



University of Colombo, Sri Lanka

UCSC *University of Colombo School of Computing*



**DEGREE OF BACHELOR OF INFORMATION TECHNOLOGY
(EXTERNAL)**

Academic Year 2025 — 1st Year Examination — Semester I

IT1406 — Introduction to Programming

Multiple Choice Question Paper

(2 Hours)

Important Instructions

- The duration of the paper is **2 Hours**.
- The medium of instructions and questions is English.
- This paper has **40 questions** on **12 pages**. Answer **all** questions.
- All questions are of the **MCQ** (Multiple Choice Questions) type.
- Each question will have 5 (**five**) choices with **ONLY ONE** correct answer.
- This paper consists of 100 marks and all the questions will carry equal marks.
- Answers should be marked on the **special answer sheet** provided.
- Note that questions appear on both sides of the paper. If a page or part of a page is not printed, please inform the supervisor/invigilator immediately.
- Mark the correct choices on the question paper first and then transfer them to the given answer sheet which will be machine marked. **Please completely read and follow the instructions given on the other side of the answer sheet before you shade your correct choices.**
- Any electronic device capable of storing and retrieving text, including electronic dictionaries, smartwatches, and mobile phones, is not allowed.
- Calculators are **not** allowed.
- *All Rights Reserved.* This question paper can NOT be used without proper permission from the University of Colombo School of Computing.

1). Which of the following statements about *Java programming language* is correct?

- a). It is a strongly typed language.
- b). It allows multiple inheritance of classes.
- c). It automatically converts conflicting types of parameters.
- d). It does not support object-oriented programming concepts.
- e). It allows two local variables with the same name in the same or enclosing scopes.

2). Which one of the following indicates a correct identifier in Java?

- a). 4th_number
- b). number-4th
- c). _4th_number
- d). #4th_number
- e). _4th/number

3). Which of the following indicates a comment in Java?

- a). #HelloWorld
- b). /**HelloWorld*/
- c). **/HelloWorld/**
- d). */HelloWorld*/
- e). */HelloWorld/*

4). Which of the following statements about *Java Access Modifiers* is correct?

- a). If no access modifier is specified, the member is private by default.
- b). The private modifier is applicable only in the context of inheritance.
- c). The main() method must always be declared with the protected modifier.
- d). The protected modifier is relevant only when inheritance is involved.
- e). A public class member can only be accessed by other members of the same class.

5). Identify the possible value for __ (A) __ and the correct type of identifier for __ (B) __, respectively.

```
1 class IdApp {
2     public static void main(String args[]) {
3         byte b = __ (A) __;
4         char c = 'd';
5         __ (B) __ result = (b/ c);
6         System.out.println("result = " + result);
7     }
8 }
```

- a). 158, int
- b). -127, int
- c). -127, char
- d). 158, char
- e). 158, byte

- 6). To search for the variable `val` in the 2D array below, what conditions should be placed at `__(A)__` and `__(B)__` in the code segment, respectively?

```

1 class NumApp {
2     public static void main(String args[]) {
3         int[][] num = {{2, 4, 6}, {8, 10, 12}};
4         int val = 1;
5         boolean found = false;
6         int i = 0;
7         search:
8         do {
9             int j = 0;
10            do {
11                if (num[i][j] == val) {
12                    found = true;
13                    break search;
14                }
15            } while (__(A)__);
16        } while (__(B)__);
17        System.out.println(found ? "Found" : "Not found"); } }

```

- a). `i < num.length, j < num[i].length`
- b). `j < num[i].length, i < num.length`
- c). `++i < num.length, ++j < num[i].length`
- d). `j++ < num[i].length, i++ < num.length`
- e). `++j < num[i].length, ++i < num.length`

- 7). Which of the following statements correctly describes the usage of a **repetition** according to the Structure Theorem?

- a). WHILE loop is an example of a counted loop in repetition.
- b). DO WHILE is an example of a trailing-decision loop in repetition.
- c). The CASE control structure is a way of expressing repetition in Java.
- d). Repetition is represented by the keywords IF, THEN, ELSE, and ENDIF.
- e). REPEAT...UNTIL is an example of a leading-decision loop in repetition.

- 8). Which of the following statements correctly describes jump statements in Java?

- (I) A *Break* exits a loop immediately, bypassing the remaining iterations.
- (II) A *Return* exits the current method and returns control to the caller.
- (III) A *Continue* skips the rest of the current iteration and proceeds with the next iteration.

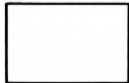
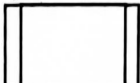
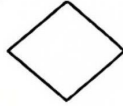
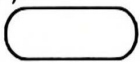
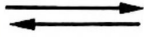
- | | | |
|--------------------|-----------------------|-------------------|
| a). I only | b). II only | c). I and II only |
| d). I and III only | e). All I, II and III | |

9). Which of the following statements correctly describe the *static* keyword in Java?

- (I) Static methods cannot use the `this` or `super` keywords to refer to instance objects.
- (II) Only methods can be declared as static.
- (III) Static methods can directly call only other static methods without an object reference.

a). I only	b). II only	c). I and II only
d). I and III only	e). All I, II, and III	

10). Which of the following gives the correct *flowchart symbols* for the examples I to IX, respectively?
Note that a symbol may be used more than once.

I. Start II. Input N III. sum = 0 IV. i = 1 V. While i <= N VI. sum = sum + i VII. i = i + 1 VIII. Print sum IX. End	Flowchart symbol			
	A). 	B). 	C). 	D). 
	E). 	F). 	G). 	H). 

a). F,C,B,B,D,B,B,C,F	b). E,C,B,B,D,B,B,C,E	d). E,C,A,A,F,A,A,C,E
c). F,C,A,A,D,A,A,C,F	e). F,C,A,A,G,A,A,C,F	

11). Which of the following correctly gives the return types of the indicated two methods, respectively?

```

1 class AreaApp {
2     float width,height;
3     double depth;
4     _____ volPrint() {
5         System.out.print("Volume is ");
6     }
7     _____ volume() {
8         return width * height * depth;
9     }
}

```

a). void,double	b). void,float	c). float,double
d). double,float	e). void,void	

12). Which of the following correctly describes **Object Orientation** in Java?

- a). A subclass inherits members but cannot access private ones.
- b). Polymorphism allows methods to have multiple return types.
- c). Encapsulation combines code and data but does not restrict access.
- d). An inner class instance can only be created within its outer class.
- e). Inheritance lets one object directly acquire another object's properties.

13). Which of the following statements are correct with respect to **method overriding** in Java?

- (I) A method in a subclass has the same name and type signature as a method in its superclass.
- (II) Method overriding supports building a generic interface for multiple methods in polymorphism.
- (III) When an overridden method is called on a subclass object, it will always execute the version defined in the superclass.

- | | | |
|--------------------|-----------------------|-------------------|
| a). I only | b). II only | c). I and II only |
| d). I and III only | e). All I, II and III | |

14). Which of the following statements correctly describes **abstract classes** in Java?

- a). Abstract classes cannot have constructors.
- b). Abstract classes cannot contain fields (variables).
- c). Abstract classes can be used to create instances (objects).
- d). Abstract classes can be used to declare object references.
- e). Abstract classes cannot include any method implementations.

15). Which of the following statements about **interfaces** in Java is correct?

- a). A class can implement only one interface.
- b). Interfaces resolve methods at compile time.
- c). Interface methods must be declared protected.
- d). Method signatures must exactly match the interface definition.
- e). If a class does not implement all interface methods, it must be private.

16). Which of the given statements about the code segment below is correct?

```
1 enum Day {  
2     MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY}
```

- a). Day can act as a superclass.
- b). The type of MONDAY is MONDAY.
- c). Day can be initiated using a new keyword.
- d). MONDAY, TUESDAY, WEDNESDAY are called enumeration constants.
- e). MONDAY, TUESDAY, WEDNESDAY are private, static final members of Day.

17). Which of the following correctly describes **type wrappers** in Java?

- a). Auto-boxing creates objects using the new keyword.
- b). `int b = obj.intValue();` is an example of auto-unboxing.
- c). `Integer obj = new Integer(100);` is an example of auto-boxing.
- d). Auto-boxing occurs when an object is converted to a primitive.
- e). Type wrapper classes encapsulate primitive values inside objects.

18). If $b = 56$, $c = 3$, and $d = 255$, what is the output of the following code segment?

```
int a = (b >> c) | d; System.out.print(a);
```

- | | | |
|---------|---------|--------|
| a). 3 | b). 7 | c). 56 |
| d). 255 | e). 511 | |

19). According to the given specifications, if the output is 5226, what should be __(A)___?

```
1 class ArrApp {  
2     public static void main(String[] args) {  
3         int [][][] x = {__(A)___};  
4         System.out.print(x[0][1][0]);  
5         System.out.print(x.length);  
6         System.out.print(x[1].length);  
7         System.out.print(x[1][1][1]);  
8     }  
9 }
```

- a). $\{\{5,7,8\},\{\underline{5},10\}\},\{\{32,56\}\}$
- b). $\{\{5,9,8\},\{\underline{5},10\}\},\{\{32,56\}\}$
- c). $\{\{5,7,8\},\{\underline{5},10\}\},\{\{32,56\},\{\underline{1},6\}\}$
- d). $\{\{5,7,8\},\{5,10\}\},\{\{32,56\},\{1,6\},\{2,3\}\}$
- e). $\{\{5,9,8\},\{5,10\}\},\{\{32,56\},\{1,6\},\{2,3\}\}$

20). Which of the following statements correctly describe *module cohesion*?

- (I) Higher closeness of elements lowers the module's cohesion.
- (II) It measures how closely the statements in a module are related.
- (III) Modules with high cohesion are considered good due to their internal strength.

- | | | |
|---------------------|-----------------------|-------------------|
| a). I only | b). II only | c). I and II only |
| d). II and III only | e). All I, II and III | |

21). Consider the following steps:

```
1 I). Connection con = DriverManager.getConnection("jdbc:mysql://  
localhost:3306/dbname", "root", "password"  
2 II). Statement stmt = con.createStatement  
3 III). con.close();  
4 IV). stmt.executeQuery("SELECT * FROM tablename");  
5 V). Class.forName("com.mysql.cj.jdbc.Driver
```

Which of the following correctly shows the steps of the *JDBC workflow*?

- | | | |
|-------------------|-------------------|-------------------|
| a). I,II,IV,V,III | b). I,II,V,IV,III | c). V,I,II,IV,III |
| d). V,I,IV,II,III | e). I,II,III,IV,V | |

22). Identify the missing object-oriented programming concept in the following code segment.

```
1 interface Pet {}  
2 abstract class Animal {  
3     String name;  
4     Animal(String name) {  
5         this.name = name; }  
6     public abstract void sound();  
7 }  
8 class Dog extends Animal implements Pet {  
9     Dog(String name) {  
10         super(name); }  
11  
12     @Override  
13     public void sound() {  
14         System.out.println(name + " says: Woof!");  
15     }  
16 }  
17 public class HelloApp {  
18     public static void main(String[] args) {  
19         Animal[] pets = { new Dog("Jimmy") };  
20         for (Animal a : pets) {  
21             a.sound();  
22         }  
23     }  
24 }
```

- | | | |
|-----------------|-------------------|------------------|
| a). Abstraction | b). Encapsulation | c). Polymorphism |
| d). Inheritance | e). Interface | |

23). Which of the given answer choices contain the suitable replacements for X,Y and Z in the following statements?

- (I) _____X_____ represents growable and writeable character sequences.
(II) _____Y_____ reads formatted input and converts it into its binary form.
(III) _____Z_____ is not thread-safe.

- a). X -StringBuilder , Y - StringBuffer, Z - Scanner
- b). X - StringBuffer, Y -StringBuilder , Z - Scanner
- c). X - Scanner, Y - StringBuilder, Z - StringBuffer
- d). X - StringBuilder, Y - Scanner, Z - StringBuffer
- e). X - StringBuffer , Y - Scanner, Z - StringBuilder

24). Which of the following statements correctly describe **collections** in Java?

- (I) ArrayList supports dynamic resizing.
(II) HashSet allows retrieval of elements by closest match.
(III) LinkedList, HashSet and HashMap are examples of collections.

- | | | |
|--------------------|-----------------------|-------------------|
| a). I only | b). II only | c). I and II only |
| d). I and III only | e). All I, II and III | |

For questions 25 to 30, select the suitable replacement for each blank.

25). _____ method defines the actions to perform before an object is destroyed.

- | | | |
|---------------------|-----------------------|----------------|
| a). getMessage() | b). printStackTrace() | c). finalize() |
| d). getStackTrace() | e). getCause() | |

26). _____ keyword prevents a method from being overridden.

- | | | |
|---------------|------------|-------------|
| a). final | b). void | c). private |
| d). protected | e). public | |

27). _____ allows a class member to be accessed outside its package, but only by direct sub-classes.

- | | | |
|-------------|---------------|------------|
| a). private | b). protected | c). public |
| d). void | e). abstract | |

28). _____ is called to get a string representation of another data type.

- | | | |
|---------------|---------------------|----------------|
| a). valueOf() | b). regionMatches() | c). getChars() |
| d). charAt() | e). toCharArray() | |

29). _____ class extends AbstractSequentialList and implements the List, Deque, and Queue interfaces.

- | | | |
|-------------------|------------------|-------------|
| a). PriorityQueue | b). NavigableSet | c). TreeSet |
| d). HashSet | e). LinkedList | |

30). _____ is a runtime error thrown when accessing a method, field, or element on a non-existent object.

- | | | |
|-------------------------|--------------------------|---------------------|
| a). ArithmeticException | b). ParseException | c). SocketException |
| d). ClassCastException | e). NullPointerException | |

For questions from 31 to 40, select the execution outputs of the given code segments.

31).

```
public class CharApp {  
2   public static void main (String args[]){  
3       char ch1 ='X';  
4       System.out.println(ch1++);  
5   }  
6 }
```

- | | | |
|--------|--------|---------|
| a). X | b). Y | c). X++ |
| d). X2 | e). XX | |

32).

```
public class ChApp {  
2   public static void main (String args[]){  
3       char ch1='A', ch2='B';  
4       if (ch1=='A'){  
5           ch2 ='G';  
6           ch2+=2;  
7           System.out.print(ch2);  
8       }  
9       System.out.print(ch2);  
10      }  
11 }
```

- | | | |
|---------|---------|--------|
| a). II | b). IB | c). BB |
| d). B2B | e). G2B | |

```

33). public class HelloApp {
2     public static void main(String[] args) {
3         String sentence = "ProgrammingIsFun";
4         char ch = sentence.charAt(sentence.length() - 5);
5         System.out.println(ch);}}

```

- | | | |
|-------|---------|-------|
| a). a | b). r | c). s |
| d). I | e). " " | |

```

34). public class NumsApp {
2     public static void main(String[] args) {
3         int[] nums = {0, 13, 2, 32, 65, 76, 24, 45};
4         int sum=0;
5         for (int n : nums) {
6             if (n%2==0) continue;
7             if (n%3==0) break;
8             if (n == 24) return;
9             System.out.print(n +sum);
10        }
11        System.out.print(nums.length);
12    }
13 }

```

- | | | |
|-----------|-------------|---------|
| a). 0 | b). 8 | c). 138 |
| d). 13658 | e). 1365458 | |

```

35). class Shape {
2     int area(int length, int width) {
3         return length * width;}
4     String type() {
5         return "Shape";
6     }
7 }
8 class Rectangle extends Shape {
9     int area(int length, int width) {
10        return length * width * 2;}
11    String type() {
12        return "Rectangle";
13    }
14 }
15 public class GeoApp {
16     public static void main(String[] args) {
17         Shape obj = new Rectangle();
18         System.out.println(obj.area(4, 5)+" "+obj.type());
19     }
20 }

```

- | | | |
|------------------|--------------------|------------------|
| a). 20 Rectangle | b). 20 Shape | c). 40 Rectangle |
| d). 40 Shape | e). Compiler Error | |

```

36) class LettApp {
2   public static void main(String[] args) {
3       try {int a = 34;
4           int b = 0;
5           try {
6               int[] arr = {1,2,3,4,5};
7               System.out.println(arr[5]);
8           }
9           catch (ArrayIndexOutOfBoundsException ex) {
10              System.out.print("A");
11          }
12          System.out.print(b / a);
13      }
14      catch (ArithmeticException ex) {
15          System.out.print("B");
16      }
17      catch (ArrayIndexOutOfBoundsException ex) {
18          System.out.print("C");
19      }
20      finally {
21          System.out.print("D");
22      }
23  }
24  }

```

- | | | |
|----------|----------|---------|
| a). A0D | b). ABC | c). ABD |
| d). A0CD | e). ABCD | |

```

37) import java.util.*;
2 class StrApp {
3     public static void main(String[] args) {
4         ArrayList<String> words = new ArrayList<String>();
5         words.add("cat");
6         words.add("tree");
7         String result = ""
8         for (String w : words) {
9             if (w.length() <= words.get(0).length()) {
10                result = result + w.length();
11            } else {result = result + w.substring(0, 2);
12            }
13        }
14        System.out.println(result);
15    }
16 }

```

- | | | |
|----------|----------|---------|
| a). 5 | b). 2tr | c). 3tr |
| d). 2trc | e). catr | |

```

38) public class NewApp {
2   public static void main(String[] args) {
3       int[] numbers = {5, 12, 7, 23, 15};
4       int i = numbers.length, y = 0;
5       do {i--;
6           y = (numbers[i] % 2 == 0) ? y + 2
7             : (numbers[i] % 5 == 0) ? y + 5 : y - 1;
8           System.out.print(y);
9       } while (i > 0);
10  }}

```

- | | | |
|-------------|--------------|------------|
| a). 52-1-15 | b). 54327 | c). 576510 |
| d). 543510 | e). 51272315 | |

```

39) public class FoodApp {
2   public static void main(String[] args)
3       String[] items = {"Rice", "Bread", "Apple", "orange"};
4       for (int i = 0; i < items.length - 1; i++) {
5           for (int j = i + 1; j < items.length; j++) {
6               if (items[i].compareTo(items[j]) < 0) {
7                   String temp = items[i];
8                   items[i] = items[j];
9                   items[j] = temp;
10              }
11          }
12      }
13      for (String item : items) {
14          System.out.print(item + " ");
15      }
16  }

```

- | | | |
|-----------------------------|-----------------------------|-----------------------------|
| a). Apple bread Rice orange | b). Apple Rice bread orange | c). orange bread Rice Apple |
| d). Rice bread Apple orange | e). Rice orange bread Apple | |

```

40) public class CaseApp {
2   public static void main(String[] args) {
3       String[] ops = {"add", "mul", "add"};
4       int a = 10, b = 5, result = 0;
5       for (String op : ops) {
6           switch (op) {
7               case "add":
8                   result = result + a + b;
9                   break;
10              case "mul":
11                  result = result + a * b;
12                  break;
13          }
14          System.out.print(result);
15      }
16  }

```

- | | | |
|------------|-----------------|--------|
| a). 0 | b). 15 | c). 80 |
| d). 156580 | e). 01050500105 | |
